UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

CENTRO UNIVERSITARIO DEL NORTE (CUNOR)

FACULTAD DE INGENIERIA ICS

SECUREFINANCE ERP

Documentación técnica y funcional

INTEGRANTES
Kenneth Christian Arnaldo Tellez Castillo
José Miguel de León Aguilar
Edward Rene Winter Tiu
Ruben Adolfo Herrera Morales

Curso: Base de Datos 2
Cobán, Alta Verapaz
2026

Índice

1. Introducción 3
2. Objetivos 3
3. Alcance 4
4. Descripción general del sistema 4
5. Tecnologías utilizadas 5
6. Arquitectura del sistema 6
7. Modelo de datos 7
8. Diagrama entidad-relación 8
9. Seguridad 11
10. Control de acceso basado en roles (RBAC) 12
11. Facturación 13
12. Transacciones ACID y concurrencia 14
13. Auditoría 15
14. Histórico de ventas 16
15. Interfaz de usuario 16
16. Instalación 17
17. Pruebas automatizadas 19
18. Matriz de pruebas 20
19. Manual de usuario 21
20. Manual técnico 21
21. Limitaciones 22
22. Conclusiones 22
23. Recomendaciones 23
24. Evidencias y anexos 23

1. Introducción

SecureFinance ERP es un sistema académico de gestión financiera desarrollado por un equipo de cuatro estudiantes de Ingeniería en Sistemas del Centro Universitario del Norte (CUNOR), como proyecto final del curso Base de Datos 2 de la Universidad de San Carlos de Guatemala. El sistema combina un backend en Node.js/Express, vistas EJS con Bootstrap, y una base de datos Microsoft SQL Server que concentra la seguridad, la lógica transaccional y la auditoría del negocio.

A diferencia de un ERP comercial, el alcance del proyecto se centra deliberadamente en demostrar, con profundidad, un conjunto acotado de temas propios del curso: autenticación con hashing y sal individual por usuario, control de acceso basado en roles (RBAC), transacciones ACID con control de concurrencia explícito, y auditoría automática mediante triggers DML. La aplicación web es un cliente delgado: toda validación de negocio, cálculo financiero y regla de integridad reside en procedimientos almacenados, funciones y restricciones de SQL Server.

Este documento describe fielmente el sistema tal como existe en la rama main del repositorio oficial al momento de esta entrega. No se documentan funcionalidades planificadas ni pendientes salvo cuando se indican expresamente como limitaciones conocidas.

2. Objetivos

2.1 Objetivo general

Diseñar, implementar y documentar un sistema ERP académico que demuestre el uso correcto de seguridad, integridad, transacciones ACID, control de concurrencia y auditoría en Microsoft SQL Server, integrado con una aplicación web Node.js que actúa únicamente como capa de presentación y orquestación.

2.2 Objetivos específicos

- Implementar autenticación con hashing SHA2_512 y sal aleatoria individual de 32 bytes por usuario, sin exponer contraseñas ni hashes fuera de SQL Server.
- Implementar control de acceso basado en roles (RBAC) mediante las tablas Usuario_Rol y Rol_Permiso, validado tanto en el procedimiento de autenticación como en cada ruta protegida de la aplicación.
- Garantizar la atomicidad de la venta multiproducto mediante un procedimiento almacenado transaccional único (sp_ProcesarVentaTransaccional), con bloqueo explícito de stock y precio hasta el COMMIT.
- Registrar de forma automática, mediante triggers, los cambios de precio/stock y eliminaciones de productos, así como la emisión de facturas, conservando valores anteriores y nuevos.
- Aplicar el principio de mínimo privilegio a la cuenta de aplicación securefinance_app, restringiendo su acceso a EXECUTE sobre procedimientos específicos, sin conceder roles amplios ni acceso directo a tablas.
- Verificar el comportamiento del sistema mediante pruebas automatizadas de Node.js y suites SQL independientes, documentando resultados reales y no supuestos.

3. Alcance

El proyecto cubre el ciclo funcional mínimo de un módulo de ventas con control financiero: autenticación de usuarios, gestión de sesión, facturación multiproducto con control de stock, generación de movimiento de caja, consulta de auditoría (accesos y cambios DML) e histórico de ventas con filtros. La interfaz web cubre cuatro pantallas: inicio de sesión, panel de trabajo (dashboard), facturación y auditoría.

Quedan fuera del alcance de esta entrega: el flujo completo de recuperación y cambio de contraseña (existen el esquema Token_Recuperacion y el indicador DebeCambiarPassword, pero sin una interfaz que obligue al cambio), la administración de usuarios/roles/permisos desde la interfaz web (se realiza mediante scripts SQL), el despliegue en producción con HTTPS, y el reemplazo del almacén de sesiones en memoria por uno persistente. Estos puntos se detallan en la sección de Limitaciones.

3.1 Equipo de trabajo y responsabilidades

El equipo del proyecto y sus responsabilidades principales, confirmadas contra la estructura real de commits y módulos del repositorio, son:

Integrante	Responsabilidad principal
Kenneth Christian Arnaldo Téllez Castillo	Arquitectura base del proyecto, estructura común, seguridad, usuarios, roles, permisos, autenticación (sp_Login, sp_RegistrarUsuario, sp_ObtenerPermisosUsuario), sesiones web, bitácora de accesos e integración general del sistema (Node.js, Express, capas de la aplicación e interfaz corporativa final).
Edward Rene Winter Tiu	Tablas de negocio (Cliente, Producto, Factura, DetalleFactura, MovimientoCaja), venta multiproducto y facturación transaccional (sp_ProcesarVentaTransaccional), control de stock y transacciones ACID con bloqueo de concurrencia.
Jose Miguel de León Aguilar	Auditoría DML mediante triggers, bitácora de transacciones (Bitacora_Transacciones), funciones de cálculo y consulta (fn_CalcularIVA, fn_CalcularSubtotal, fn_ConsultarAuditoria, fn_ObtenerHistoricoVentas) y procedimientos de reporte/histórico de ventas.
Rubén Adolfo Herrera Morales	Documentación técnica, diagrama entidad-relación final, matriz de pruebas, manual de usuario, recopilación de evidencias y documento final de entrega.

> *Nota: el historial de commits de la rama main confirma ramas de trabajo diferenciadas por integrante (feature/edward-transacciones, feature/jose-auditoria, feature/ui-polish) integradas por Kenneth, quien además realizó la consolidación final de la base de datos y de la interfaz.*

4. Descripción general del sistema

SecureFinance ERP opera bajo el nombre comercial interno "Financial Operations Desk". Un usuario autenticado accede a un panel de trabajo que muestra únicamente los módulos permitidos por sus permisos RBAC: Facturación (permiso VENTAS_REGISTRAR) y Auditoría (permiso AUDITORIA_CONSULTAR). El módulo de Facturación permite seleccionar un cliente activo, agregar uno o varios productos con cantidades, y procesar la venta en una sola operación atómica contra SQL Server. El módulo de Auditoría permite consultar tres vistas de trazabilidad: bitácora de accesos, auditoría de operaciones DML (cambios de producto y emisión de facturas) e histórico de ventas, todas con filtros por fecha y otros criterios.

La aplicación Node.js no contiene lógica de negocio: valida forma (tipos, longitudes, CSRF) y delega toda decisión —existencia de clientes o productos, suficiencia de stock, cálculo de IVA y totales, registro de auditoría— a procedimientos almacenados y triggers de SQL Server. Esta separación es intencional y es el eje central del curso: la base de datos es la autoridad de integridad y seguridad, no la aplicación web.

5. Tecnologías utilizadas

Las siguientes tecnologías fueron verificadas directamente contra package.json, server.js y los scripts de base de datos del repositorio; no se listan herramientas no utilizadas en el proyecto real.

5.1 Backend

- **Node.js** (motor de ejecución; package.json exige "engines": { "node": ">=24" })
- **Express 5.2** — enrutamiento, middleware y manejo de errores centralizado (server.js).
- **express-session 1.19** — sesión de usuario con MemoryStore (uso exclusivamente académico/local).
- **mssql 12.7** — driver de conexión a SQL Server, con parámetros tipados y Table-Valued Parameters (TVP).
- **dotenv 18** — carga de variables de entorno desde .env (no versionado).

5.2 Frontend

- **EJS 6** — motor de plantillas del lado del servidor (login, dashboard, facturación, auditoría).
- **Bootstrap 5.3** — servido localmente desde node_modules/bootstrap/dist vía la ruta /vendor/bootstrap.
- **CSS propio** (src/public/css/styles.css) — identidad visual corporativa "Financial Operations Desk" con variables de color petróleo/cobre/marfil.
- **JavaScript del navegador** (src/public/js/facturacion.js) — construcción de la venta en memoria, cálculo visual de IVA en enteros (BigInt, centavos) y envío por fetch con token CSRF.

5.3 Base de datos

- Microsoft SQL Server 2016 SP1 o posterior (requisito indicado en el README del proyecto, por el uso de CREATE OR ALTER).

5.4 Seguridad

- Hashing SHA2_512 con sal individual de 32 bytes generada con CRYPT_GEN_RANDOM.
- Procedimientos almacenados con parámetros tipados (sin concatenación de SQL).
- RBAC de tres niveles (Usuario → Rol → Permiso).
- Sesiones de servidor con cookie httpOnly y sameSite=lax.
- Principio de mínimo privilegio SQL para la cuenta de aplicación securefinance_app.

6. Arquitectura del sistema

El sistema sigue una arquitectura en capas estrictamente unidireccional: el navegador solo conoce HTTP y HTML; Express enruta y protege por sesión/permiso; los controladores validan forma; los servicios traducen a

llamadas SQL tipadas; y SQL Server concentra toda regla de negocio, integridad y auditoría. Ninguna capa de Node.js construye sentencias SQL dinámicas: todas las operaciones se ejecutan mediante procedimientos almacenados invocados con parámetros tipados (pool.request().input(...).execute(...)).

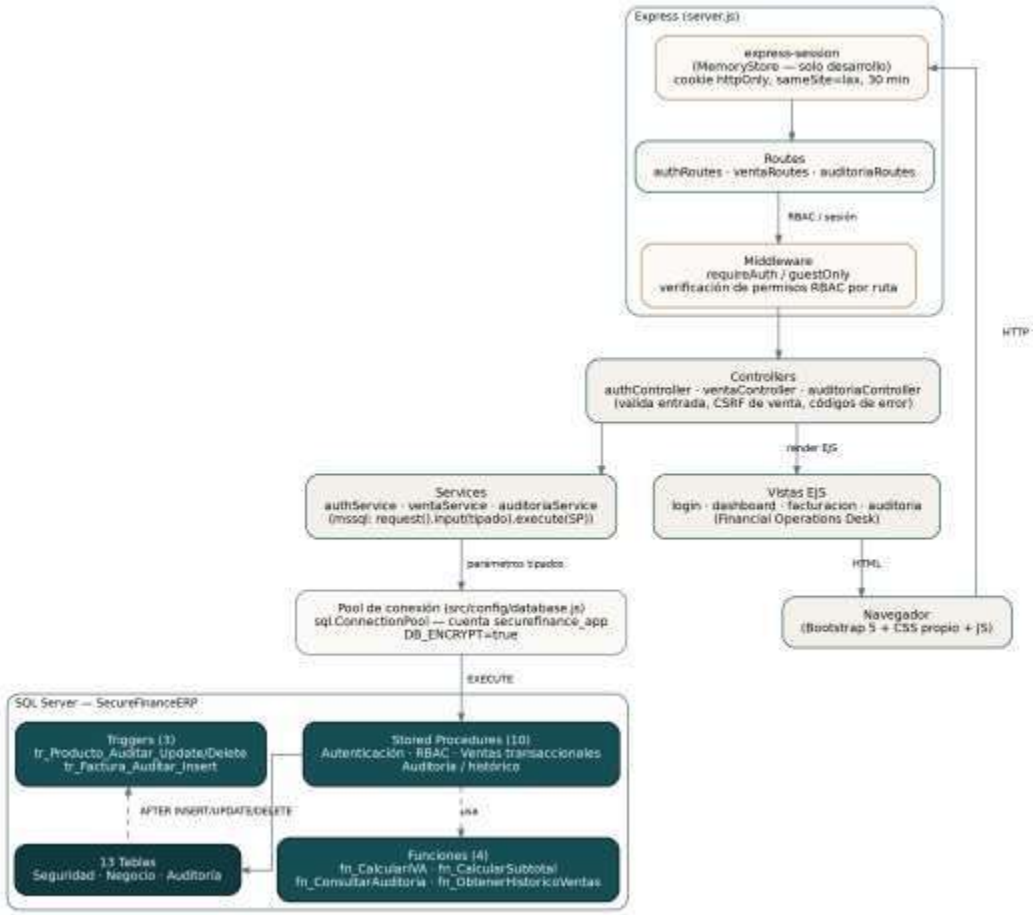


Figura 1. Arquitectura por capas de SecureFinance ERP.

6.1 Sesiones y middleware

server.js configura express-session con MemoryStore (documentado como exclusivo para desarrollo académico), cookie httpOnly, sameSite=lax y expiración de 30 minutos. El middleware requireAuth redirige a /login si no existe sesión; guestOnly evita que un usuario ya autenticado vuelva a ver el login. Cada grupo de rutas (ventas, auditoría) añade además una verificación de permiso RBAC específico antes de delegar al controlador; si el permiso no está presente responde 403 sin ejecutar lógica adicional.

6.2 Separación por capas

Capa	Responsabilidad
Routes (src/routes)	Define endpoints HTTP, aplica middleware de sesión/permiso y desactiva caché de respuesta.
Controllers (src/controllers)	Valida forma de la solicitud (tipos, rangos, CSRF), traduce errores SQL a mensajes seguros y arma la respuesta HTTP/JSON.

Capa	Responsabilidad
Services (src/services)	Única capa que llama a mssql; construye parámetros tipados y Table-Valued Parameters; no contiene reglas de negocio.
Config (src/config/database.js)	Gestiona el pool de conexión único (sql.ConnectionPool), valida variables de entorno y prohíbe explícitamente la cuenta sa.
Views (src/views)	Plantillas EJS que renderizan HTML a partir de los datos ya validados por el controlador.
SQL Server (database/)	Procedimientos, funciones, triggers y restricciones: única autoridad de integridad, cálculo financiero y auditoría.

7. Modelo de datos

La base de datos SecureFinanceERP contiene 13 tablas agrupadas en tres módulos —Seguridad, Negocio y Auditoría— más un tipo de tabla definido por el usuario (TipoDetalleVenta) usado como parámetro de tabla (TVP) para enviar el detalle de una venta en una sola llamada. A continuación se documenta cada tabla tal como está definida en los scripts 02, 05 y 07 de la carpeta database/.

7.1 Módulo de Seguridad

Tabla	PK	FK	Propósito / reglas de integridad relevantes
Usuario	UsuarioId	—	Identidad y credenciales. PasswordHash VARBINARY(64) y Salt VARBINARY(32) con longitud forzada por CHECK; NombreUsuario, Correo y Salt son UNIQUE; DebeCambiarPassword indica cambio pendiente (sin flujo de UI).
Rol	RolId	—	Catálogo de roles (Administrador, Cajero, Auditor). Nombre UNIQUE.
Permiso	PermisoId	—	Catálogo de permisos (Codigo UNIQUE), p. ej. VENTAS_REGISTRAR.
Usuario_Rol	(UsuarioId, RolId)	Usuario, Rol	Asignación N:M de roles a usuarios, con fecha de asignación.
Rol_Permiso	(RolId, PermisoId)	Rol, Permiso	Asignación N:M de permisos a roles.
Token_Recuperacion	TokenRecuperacionId	Usuario	Esquema para recuperación de contraseña (TokenHash, expiración, invalidación). No integrado a una interfaz de cambio de contraseña.
Bitacora_Acceso	BitacoraAccesoId	Usuario (nullable)	Registra cada intento de login con su resultado; UsuarioId es NULL únicamente cuando el resultado es USUARIO_INEXISTENTE (forzado por CHECK).

7.2 Módulo de Negocio

Tabla	PK	FK	Propósito / reglas de integridad relevantes
Cliente	ClienteId	—	Clientes de venta. NIT UNIQUE; Activo controla si puede facturarse.
Producto	ProductoId	—	Catálogo de productos con Precio (sin IVA) y Stock; ambos con CHECK >= 0.
Factura	FacturaId	Cliente, Usuario	Cabecera de venta; CHECK obliga Total = Subtotal + IVA; Estado fijo en EMITIDA (no hay anulación).
DetalleFactura	DetalleFacturaId	Factura, Producto	Líneas de venta; UNIQUE(FacturaId, ProductoId) impide productos repetidos en una factura; CHECK valida Subtotal = PrecioUnitario × Cantidad.
MovimientoCaja	MovimientoCajaId	Factura (UNIQUE)	Ingreso de caja asociado 1 a 1 a la factura; TipoMovimiento fijo en INGRESO.

7.3 Módulo de Auditoría

Tabla	PK	FK	Propósito / reglas de integridad relevantes
Bitacora_Transacciones	BitacoraTransaccionId	—	Registro DML genérico alimentado por triggers; guarda ValorAnterior/ValorNuevo en JSON (FOR JSON PATH), TablaAfectada, Operacion (INSERT/UPDATE/DELETE) y metadatos de conexión.

> *El tipo de tabla dbo.TipoDetalleVenta (ProductoId INT, Cantidad INT) no es una tabla persistente: es un Table-Valued Parameter usado exclusivamente para enviar el detalle completo de una venta a sp_ProcesarVentaTransaccional en una sola llamada.*

8. Diagrama entidad-relación

Dado el tamaño del modelo, el ERD se presenta en tres niveles: un diagrama general de referencia y dos diagramas detallados por módulo (Seguridad/RBAC y Negocio/Facturación). Los tres se generaron con Graphviz a partir de las restricciones reales declaradas en los scripts SQL; el código fuente (.dot) se entrega en docs/diagramas/.

8.1 ERD general

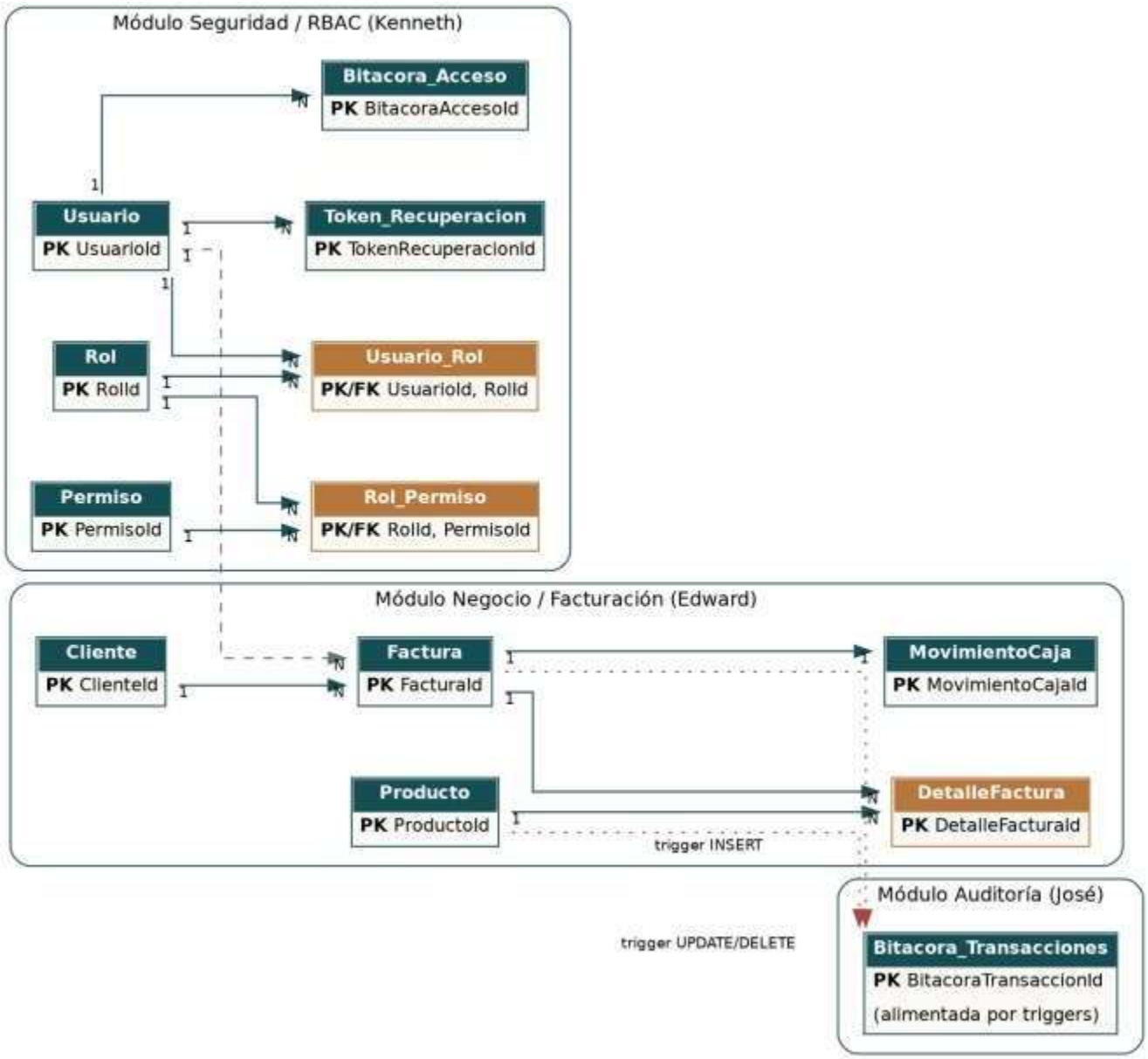


Figura 2. ERD general — los tres módulos y sus relaciones (líneas punteadas: trazabilidad de triggers).

8.2 ERD de Seguridad / RBAC

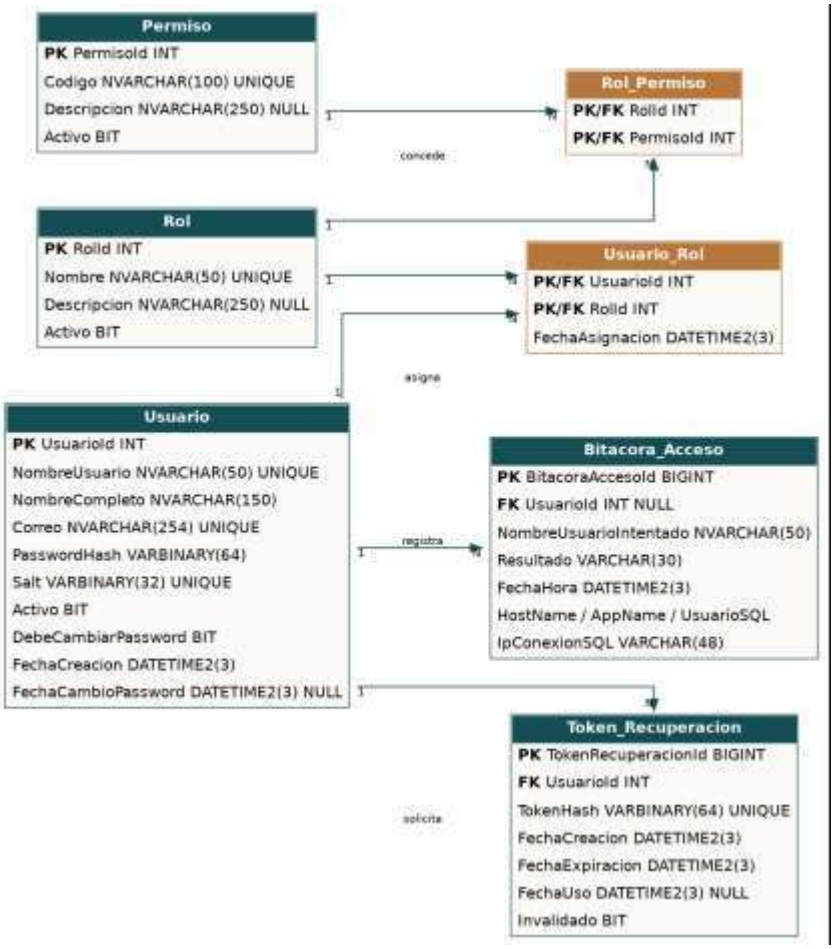


Figura 3. ERD del módulo de Seguridad: Usuario, Rol, Permiso y sus tablas puente.

8.3 ERD de Negocio / Facturación

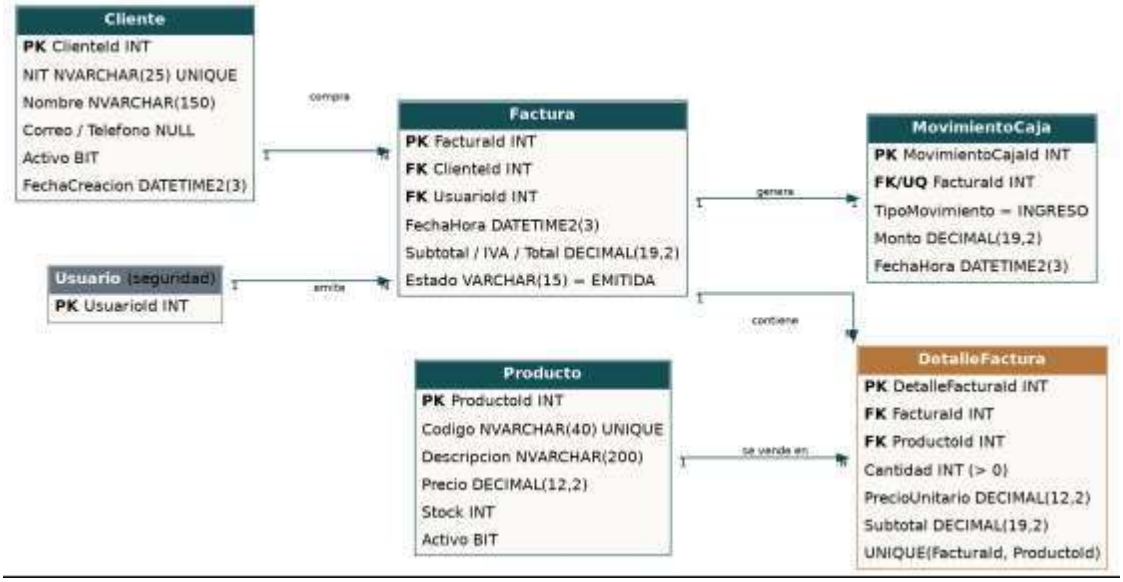


Figura 4. ERD del módulo de Negocio: Cliente, Producto, Factura, DetalleFactura y MovimientoCaja.

9. Seguridad

9.1 Registro y hashing de contraseñas

El procedimiento sp_RegistrarUsuario valida longitud de los campos (contraseña entre 1 y 256 bytes), verifica que NombreUsuario y Correo no existan previamente, genera una sal de 32 bytes con CRYPT_GEN_RANDOM(32) y calcula el hash con la fórmula única del sistema:

```
HASHBYTES('SHA2_512', CONVERT(VARBINARY(256), @Password) + @Salt)
```

El resultado, de 64 bytes, se almacena en PasswordHash; la sal se almacena en Salt, con restricción UNIQUE para impedir colisiones. Ni la contraseña ni el hash ni la sal se recortan, normalizan ni transforman en Node.js: el backend solo transporta el valor tecleado por el usuario hacia el procedimiento almacenado.

9.2 Autenticación (sp_Login)

sp_Login busca al usuario por NombreUsuario, calcula el hash entrante con la misma fórmula y sal almacenada, y lo compara contra PasswordHash. Devuelve siempre un código y un resultado textual, nunca el hash ni la sal:

Código	Resultado	Significado
0	EXITOSO	Credenciales correctas; se retornan los datos de sesión (UsuarioId, NombreUsuario, Correo, DebeCambiarPassword).
1	USUARIO_INEXISTENTE	No existe un usuario con ese NombreUsuario.
2	USUARIO_INACTIVO	El usuario existe pero Activo = 0.
3	PASSWORD_INCORRECTA	El hash calculado no coincide con el almacenado.

Cada intento —exitoso o no— se registra mediante sp_RegistrarAcceso en Bitacora_Acceso, incluyendo HostName, AppName, UsuarioSQL y la IP de la conexión SQL (que corresponde al backend, no necesariamente al navegador del usuario final). sp_Login se ejecuta en autocommit, de forma que un rollback externo también revertiría su propio registro de bitácora.

9.3 Sesión web

Tras un login exitoso, authController.login regenera el identificador de sesión (mitigación de fijación de sesión) antes de guardar la identidad del usuario, y solo entonces redirige al dashboard. El controlador consulta además sp_ObtenerPermisosUsuario para fijar en sesión la lista de códigos de permiso vigentes; estos cambios de RBAC solo se reflejan la próxima vez que el usuario inicia sesión, no en caliente.

9.4 Principio de mínimo privilegio — securefinance_app

La aplicación se conecta a SQL Server con una cuenta dedicada, securefinance_app, cuyos permisos se otorgan exclusivamente mediante database/10_AppPermissions.sql. src/config/database.js impide en tiempo de ejecución que DB_USER sea sa. La cuenta no recibe sysadmin, db_owner, db_datareader, db_datawriter, CONTROL ni permisos directos de SELECT/INSERT/UPDATE/DELETE sobre tablas: únicamente EXECUTE sobre procedimientos específicos y REFERENCES/EXECUTE sobre el tipo de tabla usado como TVP.

Área	Permisos concedidos a securefinance_app
Autenticación	EXECUTE sobre dbo.sp_Login y dbo.sp_ObtenerPermisosUsuario.
Ventas	EXECUTE sobre dbo.sp_ListarClientes, dbo.sp_ListarProductosDisponibles y dbo.sp_ProcesarVentaTransaccional.
TVP	EXECUTE y REFERENCES sobre TYPE::dbo.TipoDetalleVenta.
Auditoría	EXECUTE sobre dbo.sp_ConsultarBitacoraAcceso, dbo.sp_ConsultarAuditoriaTransacciones y dbo.sp_ConsultarHistoricoVentas.

Los procedimientos internos auxiliares (por ejemplo sp_RegistrarAcceso, invocado desde sp_Login) se ejecutan bajo la cadena de propiedad de dbo, por lo que no requieren un GRANT independiente para la cuenta de aplicación. El script no revoca privilegios preexistentes; queda bajo responsabilidad del administrador confirmar que la cuenta dedicada no herede permisos adicionales por otra vía. Ninguna contraseña real de esta cuenta se publica en este documento ni en el repositorio: solo se define en el archivo local .env, que no se versiona.

10. Control de acceso basado en roles (RBAC)

El modelo de autorización sigue la cadena Usuario, Usuario_Rol, Rol, Rol_Permiso, Permiso. sp_ObtenerPermisosUsuario resuelve, con un DISTINCT sobre el join completo, el conjunto efectivo y único de códigos de permiso de un usuario, considerando solo roles y permisos activos.

10.1 Roles y permisos reales (semillas)

Rol	Permisos asignados
Administrador	USUARIOS_ADMINISTRAR, VENTAS_REGISTRAR, AUDITORIA_CONSULTAR
Cajero	VENTAS_REGISTRAR
Auditor	AUDITORIA_CONSULTAR

> *El permiso USUARIOS_ADMINISTRAR existe en el catálogo y se asigna al rol Administrador, pero no tiene actualmente una ruta ni controlador en la aplicación web que lo consuma: la administración de usuarios/roles se realiza mediante scripts SQL. Esto se documenta como limitación, no como funcionalidad implementada.*

10.2 Aplicación del RBAC en la web

Cada grupo de rutas protegidas valida el permiso requerido antes de invocar al controlador: ventaRoutes exige VENTAS_REGISTRAR para todo el prefijo /facturacion; auditoriaRoutes exige AUDITORIA_CONSULTAR para /auditoria. Sin sesión activa, cualquier ruta protegida redirige a /login; con sesión pero sin el permiso requerido, la respuesta es 403 sin ejecutar ninguna consulta a SQL Server. El dashboard, por su parte, oculta o muestra los módulos de navegación según los permisos presentes en sesión, evitando enlaces a áreas no autorizadas.

11. Facturación

El flujo de venta multiproducto integra Cliente, Producto, Factura, DetalleFactura y MovimientoCaja en una sola operación atómica ejecutada por sp_ProcesarVentaTransaccional. La ruta GET /facturacion (protegida por sesión y por el permiso VENTAS_REGISTRAR) muestra el catálogo de clientes y productos activos junto con un token CSRF de un solo propósito almacenado en sesión (req.session.ventaCsrf); la ruta POST

/facturacion/procesar exige ese token en el encabezado X-CSRF-Token y lo compara con timingSafeEqual antes de aceptar la solicitud.

11.1 Flujo completo

- El usuario autenticado selecciona un cliente y agrega productos con cantidades desde la interfaz (src/public/js/facturacion.js), que calcula un IVA de vista previa en enteros (centavos) para evitar errores de redondeo en pantalla.
- ventaController valida estructura: cliente y usuario deben ser enteros positivos válidos; el detalle debe tener entre 1 y 100 líneas, con ProductoId y Cantidad enteros positivos y sin productos repetidos. Precios e importes enviados por el navegador se ignoran por completo: el servidor solo reenvía ProductoId y Cantidad.
- ventaService construye un parámetro de tabla (TVP) de tipo dbo.TipoDetalleVenta y ejecuta sp_ProcesarVentaTransaccional con ClienteId, UsuarioId y el detalle completo en una sola llamada.
- Dentro de una única transacción, el procedimiento valida cliente activo, usuario activo, y para cada producto —en orden ascendente por ProductoId— bloquea precio y stock, valida existencia, estado y stock suficiente, y descuenta el stock.
- SQL Server calcula Subtotal, IVA (12% sobre el subtotal global, redondeado a dos decimales) y Total, e inserta Factura, DetalleFactura y MovimientoCaja antes de hacer COMMIT.
- La respuesta exitosa (201) regresa FacturaId, fecha y los tres importes como cadenas de texto (no como número de punto flotante), para no perder centavos al convertir a JavaScript.

11.2 Errores de negocio

sp_ProcesarVentaTransaccional señaliza cada regla de negocio con un número de error THROW específico, que ventaController traduce a un mensaje seguro para el usuario sin exponer detalles internos de SQL:

Error SQL	Causa	Mensaje mostrado
52001	Detalle vacío	Agrega al menos un producto.
52002	Más de 100 líneas	La venta admite hasta 100 productos.
52003	Cantidad o ProductoId no positivos	El producto y la cantidad deben ser enteros positivos.
52004	Producto repetido en el detalle	Cada producto debe aparecer una sola vez.
52005	Cliente no existe	El cliente no existe.
52006	Cliente inactivo	El cliente está inactivo.
52007	Usuario no existe o inactivo	El usuario no existe o está inactivo.
52008	Producto no existe	Uno de los productos no existe.
52009	Producto inactivo	Uno de los productos está inactivo.
52010	Stock insuficiente	Stock insuficiente. Actualiza la página y revisa las cantidades.

11.3 Cálculo financiero (caso validado)

Con los precios DEMO originales (Teclado USB Q125.50, Mouse óptico Q75.25), la venta de 2 teclados y 3 mouse produce:

Línea	Cantidad	Precio unitario	Subtotal de línea
Teclado USB	2	Q 125.50	Q 251.00
Mouse óptico	3	Q 75.25	Q 225.75

Concepto	Monto
Subtotal	Q 476.75
IVA (12% sobre el subtotal)	Q 57.21
Total	Q 533.96

Estos valores los calcula y devuelve SQL Server; la aplicación web únicamente los muestra. El redondeo del IVA se aplica una sola vez sobre el subtotal global (ROUND(@Subtotal * 0.12, 2)), no línea por línea, para evitar diferencias de centavos por acumulación de redondeos.

12. Transacciones ACID y concurrencia

12.1 Atomicidad, consistencia, aislamiento y durabilidad

Propiedad	Cómo se garantiza en sp_ProcesarVentaTransaccional
Atomicidad	BEGIN TRANSACTION ... COMMIT envuelve validaciones, descuento de stock e inserción de Factura/DetalleFactura/MovimientoCaja. Cualquier error dentro del bloque TRY dispara THROW; el bloque CATCH ejecuta ROLLBACK TRANSACTION si XACT_STATE() <> 0. No existe un solo camino donde se inserte una factura sin su detalle y su movimiento de caja, o viceversa.
Consistencia	Las restricciones CHECK (p. ej. CK_Factura_Importes: Total = Subtotal + IVA; CK_DetalleFactura_Subtotal) y las claves foráneas garantizan que ninguna transacción pueda dejar la base en un estado que viole las reglas declaradas, incluso si el procedimiento tuviera un error de cálculo.
Aislamiento	SET XACT_ABORT ON asegura que cualquier error de tiempo de ejecución aborte y revierta toda la transacción automáticamente. El acceso a Producto usa WITH (UPDLOCK, HOLDLOCK) para que ninguna otra sesión pueda leer un valor de precio/stock "sucio" ni modificarlo hasta el COMMIT.
Durabilidad	Una vez ejecutado COMMIT TRANSACTION, los cambios son permanentes según las garantías estándar del motor de SQL Server (registro de transacciones en el log). Node.js invoca el procedimiento en autocommit y no reintenta automáticamente una venta cuyo resultado quedó incierto (p. ej. por timeout de red después del COMMIT), precisamente para no duplicar una operación ya durable.

12.2 Caso de rollback por stock insuficiente

Si dentro de una venta multiproducto una sola línea no tiene stock suficiente (error 52010), el procedimiento revierte la transacción completa: no se genera una factura parcial, no se genera un movimiento de caja parcial y no se descuenta stock de ninguno de los productos de esa venta, incluidos los que sí tenían existencia suficiente. Este comportamiento se valida explícitamente en database/tests/TransactionTests.sql y en tests/venta.test.js.

12.3 Control de concurrencia

sp_ProcesarVentaTransaccional recorre las líneas del detalle en orden ascendente por ProductoId (no en el orden en que llegaron desde el navegador), leyendo cada fila de Producto con WITH (UPDLOCK, HOLDLOCK). Esto tiene dos efectos verificados en el código:

- UPDLOCK reserva la fila para actualización desde el momento de la lectura, evitando que dos ventas concurrentes lean el mismo stock "disponible" y ambas lo descuenten como si el stock no hubiera cambiado.
- HOLDLOCK conserva ese bloqueo hasta el COMMIT o ROLLBACK de la transacción completa, de modo que precio y stock quedan fijos para esa venta durante toda su ejecución.
- El recorrido en orden ascendente por ProductoId —independiente del orden del TVP recibido— es la estrategia usada para reducir el riesgo de interbloqueo (deadlock) cuando dos ventas concurrentes comparten productos: ambas transacciones intentan adquirir los bloqueos en la misma secuencia.

> *El sistema no implementa reintentos automáticos de una venta ante un timeout: si el resultado de una operación queda incierto tras el envío, la aplicación instruye a consultar con el responsable antes de reenviar la venta, evitando así una doble facturación accidental.*

13. Auditoría

La auditoría del sistema tiene dos fuentes independientes y con propósitos distintos: Bitacora_Acceso (intentos de inicio de sesión, alimentada desde sp_Login/sp_RegistrarAcceso) y Bitacora_Transacciones (cambios DML sobre tablas de negocio, alimentada exclusivamente por triggers). Ambas se consultan desde /auditoria mediante procedimientos de solo lectura, protegidos por el permiso AUDITORIA_CONSULTAR.

13.1 Triggers implementados

Trigger	Tabla / evento	Qué registra
tr_Producto_Auditar_Update	Producto — AFTER UPDATE	Solo cuando cambia Precio o Stock (condición WHERE d.Precio <> i.Precio OR d.Stock <> i.Stock); guarda el registro completo anterior y nuevo como JSON.
tr_Producto_Auditar_Delete	Producto — AFTER DELETE	Registra el valor eliminado completo en ValorAnterior; ValorNuevo queda NULL.
tr_Factura_Auditar_Insert	Factura — AFTER INSERT	Registra la factura emitida completa en ValorNuevo; ValorAnterior queda NULL (no hay estado previo en un INSERT).

Los tres triggers insertan en Bitacora_Transacciones con TablaAfectada, Operacion (INSERT/UPDATE/DELETE), UsuarioSQL, HostName, AppName, IdentificadorRegistro (la clave del registro afectado) y los valores en JSON generados con FOR JSON PATH, WITHOUT_ARRAY_WRAPPER.

13.2 Consulta de auditoría

fn_ConsultarAuditoria es una función de tabla en línea que filtra Bitacora_Transacciones por rango de fecha, tabla y operación; sp_ConsultarAuditoriaTransacciones la envuelve y ordena los resultados por fecha

descendente. De forma análoga, sp_ConsultarBitacoraAcceso filtra Bitacora_Acceso por fecha, nombre de usuario intentado y resultado.

13.3 Qué no se audita ni se almacena

Por diseño explícito, ni Bitacora_Acceso ni Bitacora_Transacciones almacenan PasswordHash, Salt, TokenHash, contraseñas en texto claro ni ningún otro secreto: la auditoría registra metadatos de la operación (quién, cuándo, qué tabla, qué cambió), nunca credenciales.

14. Histórico de ventas

fn_ObtenerHistoricoVentas es una función de tabla en línea que une Factura con Cliente y Usuario, exponiendo Factura, Fecha, Cliente, Usuario, Subtotal, IVA y Total, con filtros opcionales de fecha inicial, fecha final y nombre de cliente (coincidencia parcial mediante CHARINDEX). sp_ConsultarHistoricoVentas la invoca y ordena el resultado por fecha y número de factura descendente, de modo que las ventas más recientes aparecen primero.

Desde la interfaz web, la pestaña de histórico dentro de /auditoria reutiliza los mismos filtros de fecha del resto del panel y agrega un filtro adicional por nombre de cliente, permitiendo verificar rápidamente que una venta procesada (por ejemplo, la del caso validado de la sección 11.3) aparece con sus importes exactos.

15. Interfaz de usuario

La interfaz adopta una identidad visual corporativa propia, "Financial Operations Desk": paleta oscura de petróleo (#164e56) y tinta (#101820) combinada con acentos de cobre (#b7773c) sobre fondos claros de marfil y papel, tipografía de sistema y una barra lateral fija de navegación. El objetivo declarado en el CSS del proyecto es transmitir seriedad operativa —similar a una mesa de control financiero— evitando un aspecto de plantilla genérica de administración.

15.1 Pantallas

Pantalla	Ruta	Contenido principal
Login	/login	Formulario de usuario y contraseña sobre fondo oscuro con retícula sutil; muestra el mensaje de error devuelto por sp_Login sin detalles técnicos.
Dashboard	/dashboard	Saludo al usuario, conteo de permisos activos, tarjetas de acceso a los módulos habilitados (Facturación y/o Auditoría según RBAC) y listado de permisos asignados.
Facturación	/facturacion	Selector de cliente, selector de producto con stock visible, tabla de líneas agregadas, resumen de subtotal/IVA/total en vivo y botón de procesar venta.
Auditoría	/auditoria	Tres bloques con métricas resumidas (accesos, eventos DML, ventas) y tres tablas filtrables: bitácora de accesos, auditoría DML e histórico de ventas.

15.2 Navegación y jerarquía visual

Una barra lateral fija (sidebar) de 238px muestra la marca, el área de trabajo activa mediante numeración tipo "00 / 01 / 02" y el usuario en sesión con acceso directo a cerrar sesión. El contenido principal usa una jerarquía tipográfica consistente (eyebrow en mayúsculas, título de página, texto secundario) y tablas de datos con densidad alta para las vistas de auditoría, donde el volumen de filas es mayor. El diseño es responsivo mediante utilidades de Bootstrap y unidades relativas, e incluye un enlace de salto al contenido ("Ir al contenido") para accesibilidad de teclado.

Este documento no profundiza en un análisis de diseño gráfico: el objetivo de esta sección es dejar constancia de que la interfaz implementada corresponde a las pantallas y módulos aquí descritos, no a un mockup aspiracional.

16. Instalación

16.1 Requisitos

- Node.js 24 o posterior y npm.
- Microsoft SQL Server 2016 SP1 o posterior (usa CREATE OR ALTER), con TCP/IP habilitado y el puerto real de la instancia.
- SQL Server Management Studio (SSMS) o sqlcmd para ejecutar los scripts.

16.2 Opción A — Instalación completa (script Full)

- Ejecutar `npm ci` para instalar las dependencias exactas del proyecto.
- Crear o configurar manualmente el login SQL externo securefinance_app (o una cuenta equivalente con los mismos permisos mínimos); ningún script crea logins ni contraseñas de servidor.
- Ejecutar database/SecureFinanceERP_Full.sql en SSMS o sqlcmd sobre una base nueva. Este script consolida los diez bloques modulares en un único archivo, sin las suites de prueba.
- Si el login se creó después del Full, ejecutar database/10_AppPermissions.sql para aplicar los permisos mínimos.
- Crear el archivo .env local a partir de .env.example, con SESSION_SECRET aleatorio de al menos 32 caracteres y las credenciales de la cuenta SQL dedicada (nunca sa).
- Ejecutar `npm run check`, luego `npm test` y luego `npm run check:db` para verificar sintaxis, pruebas con mocks y conexión real a SQL Server, en ese orden.
- Ejecutar `npm start` y abrir http://localhost:3000/login.

> *Si el login securefinance_app no existe al momento de ejecutar el Full, el script no aborta la instalación: imprime una advertencia indicando que los permisos de aplicación quedan pendientes hasta que el administrador cree el login y ejecute 10_AppPermissions.sql.*

16.3 Opción B — Instalación modular

Tras instalar dependencias y preparar el login, ejecutar en este orden exacto (la dependencia entre archivos está impuesta por las referencias FK y por los objetos que cada trigger requiere):

#	Script	Contenido
1	01_CreateDatabase.sql	Crea la base de datos SecureFinanceERP si no existe.
2	02_SecurityTables.sql	Tablas de seguridad: Usuario, Rol, Permiso, Usuario_Rol, Rol_Permiso, Token_Recuperacion, Bitacora_Acceso.
3	03_SecurityStoredProcedures.sql	sp_RegistrarUsuario, sp_RegistrarAcceso, sp_Login, sp_ObtenerPermisosUsuario.
4	04_SecuritySeedData.sql	Roles, permisos, asignaciones y usuario admin_demo.
5	05_BusinessTables.sql	Cliente, Producto, Factura, DetalleFactura, MovimientoCaja y el tipo TipoDetalleVenta.
6	05_BusinessSeedData.sql	Clientes y productos ficticios de demostración.
7	06_TransactionProcedures.sql	sp_ListarClientes, sp_ListarProductosDisponibles, sp_ProcesarVentaTransaccional.
8	07_AuditCore.sql	Tabla Bitacora_Transacciones, funciones fn_CalcularIVA/fn_CalcularSubtotal/fn_ConsultarAuditoria/fn_ObtenerHistoricoVentas y los tres procedimientos de consulta.
9	08_AuditTriggers.sql	Los tres triggers de auditoría DML sobre Producto y Factura.
10	10_AppPermissions.sql	Permisos mínimos de la cuenta securefinance_app.

database/07_AuditCore.sql requiere que Factura, Cliente y Usuario ya existan; los triggers requieren Producto, Factura y Bitacora_Transacciones. Por esta razón, la auditoría se instala después del módulo de negocio, y los triggers después del núcleo de auditoría. Los scripts 07_TransactionPermissions.sql y 09_AuditPermissions.sql se conservan en el repositorio como versiones parciales/históricas; para la instalación final se usa únicamente 10_AppPermissions.sql, que consolida todas las concesiones necesarias.

16.4 Variables de entorno (.env)

El archivo .env (no versionado) define, sin publicar secretos reales: NODE_ENV, PORT, SESSION_SECRET, y los parámetros de conexión DB_SERVER, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD, DB_ENCRYPT y DB_TRUST_SERVER_CERTIFICATE (este último solo para un certificado autofirmado en desarrollo local). La aplicación escucha por defecto en 127.0.0.1:3000.

17. Pruebas automatizadas

El proyecto define cuatro comprobaciones de Node.js y cuatro suites SQL independientes. Como parte de la preparación de este documento se ejecutaron realmente npm ci, npm run check y npm test contra el código de la rama main, con los siguientes resultados verificados (no supuestos):

Comando	Propósito	Resultado obtenido en esta verificación
npm run check	Comprobación de sintaxis JavaScript de la aplicación y de las pruebas (node --check sobre cada archivo clave).	Sin errores de sintaxis en ninguno de los 18 archivos verificados.
npm test	Pruebas de Node (node --test) de autenticación, ventas/facturación, frontend de facturación y auditoría, con mocks; no conecta a SQL Server real.	18 de 18 pruebas superadas (# pass 18, # fail 0), duración total ≈1.56 s.
npm run check:db	Verifica la conexión real a SQL Server usando la configuración local de .env.	No ejecutado en este entorno de documentación por no disponer de una instancia de SQL Server accesible; requiere .env configurado por el equipo.

17.1 Pruebas de Node por archivo

Archivo	Casos	Enfoque
tests/auth.test.js	3	Login, gestión de sesión, permisos escapados en HTML, manejo de errores y cierre de sesión, con el driver SQL simulado.
tests/venta.test.js	8	Servicio de ventas con TVP tipado; rutas HTTP de facturación (sesión, permiso VENTAS_REGISTRAR, CSRF, validación de estructura, mensajes de error de negocio, límite de tamaño de cuerpo).
tests/facturacion.test.js	3	Comportamiento del frontend: acumulación de cantidades repetidas, cálculo de IVA en centavos exactos, bloqueo de doble clic y manejo de stock insuficiente sin reintento automático.
tests/audit.test.js	4	Servicio de auditoría: filtros tipados, propagación de errores SQL, contrato de datos esperado y ruta HTTP de auditoría (sesión, permiso, 200/403).

17.2 Suites SQL

Archivo	Cubre
database/tests/SecurityTests.sql	Esquema y restricciones de seguridad: relaciones, valores por defecto, longitud de hash/sal y unicidades.
database/tests/AuthenticationTests.sql	Registro con SHA2_512, unicidad de sales, flujo completo de login, bitácora de acceso y resolución de permisos RBAC.
database/tests/TransactionTests.sql	Ventas transaccionales: cálculo de importes, control de stock y rollback ante error de negocio (incluido stock insuficiente).
database/tests/AuditTests.sql	Funciones de cálculo/consulta, disparo correcto de los tres triggers y consultas de auditoría e histórico de ventas.

Las cuatro suites SQL deben ejecutarse contra una instancia real de SQL Server después de instalar el esquema completo, con una cuenta de desarrollo autorizada y una base de pruebas sin escritores concurrentes; no están incluidas en el script Full. Sus transacciones internas revierten los datos temporales que insertan, aunque los contadores IDENTITY pueden avanzar como efecto normal de SQL Server. Por no disponer de una instancia de

SQL Server en este entorno de documentación, sus resultados no se marcan como validados en este ejercicio; deben ejecutarse y revisarse por el equipo antes de la entrega final, junto con npm run check:db.

Adicionalmente existe npm run test:sql -- ".\SQLEXPRESS", que requiere sqlcmd, autenticación de Windows y permisos para crear y eliminar una base temporal aislada; su alcance actual cubre seguridad, autenticación y ventas (incluida concurrencia y permisos transaccionales), pero no instala auditoría ni valida el script Full ni 10_AppPermissions.sql, por lo que no sustituye a las cuatro suites SQL ni a la verificación con la cuenta real de aplicación.

18. Matriz de pruebas

La matriz de pruebas completa —con precondición, entrada, resultado esperado, resultado obtenido, estado y evidencia referenciada por caso— se entrega como archivo independiente y editable: docs/Matriz_Pruebas_SecureFinance_ERP.xlsx. La tabla siguiente resume su cobertura por módulo; el estado "VALIDADO" se usa únicamente para los casos efectivamente comprobados en esta documentación (pruebas Node ejecutadas) o en el propio código fuente revisado, y "PENDIENTE" para los que requieren una instancia de SQL Server o una sesión de navegador que el equipo debe ejecutar antes de la entrega.

Módulo	Casos incluidos	Estado en esta entrega
Autenticación	Login correcto, password incorrecta, usuario inexistente, usuario inactivo, acceso sin sesión	Validado por pruebas Node (mocks); pendiente de verificación manual contra SQL Server real.
RBAC	Acceso con permiso, acceso sin permiso (403)	Validado por pruebas Node.
Ventas	Venta válida, venta multiproducto, stock insuficiente, productos duplicados, cantidad inválida	Validado por pruebas Node; caso financiero (Q533.96) validado por revisión de código y semillas reales.
ACID	Rollback completo ante stock insuficiente	Validado por pruebas Node; pendiente confirmación en TransactionTests.sql contra SQL Server real.
Auditoría	Acceso, Producto UPDATE, Producto DELETE, Factura INSERT, histórico	Lógica de triggers verificada por revisión de código; pendiente de evidencia visual contra SQL Server real.
Seguridad SQL	EXECUTE permitido a securefinance_app, SELECT directo denegado	Permisos verificados por revisión de 10_AppPermissions.sql; pendiente de comprobación en vivo con la cuenta real.
Node / calidad	npm run check, npm test (18/18)	Validado en esta documentación: ejecutado realmente, 18/18 pruebas superadas.

> *Ningún caso se marca como VALIDADO en la matriz sin evidencia concreta (ejecución real o revisión directa del código/script correspondiente). Los casos que requieren una instancia de SQL Server activa, una sesión de navegador real o una captura de pantalla se marcan explícitamente como PENDIENTE en el archivo .xlsx, con la instrucción exacta de qué ejecutar para completarlos.*

19. Manual de usuario

El manual de usuario completo, redactado en lenguaje no técnico y con capturas marcadas, se entrega como documento independiente: docs/Manual_Usuario_SecureFinance_ERP.docx (y su versión .pdf). Cubre el flujo completo desde iniciar el sistema hasta cerrar sesión: login, dashboard, facturación (selección de cliente, producto, cantidad, revisión de totales y procesamiento de la venta), consulta de auditoría con sus filtros y resultados, y cierre de sesión.

20. Manual técnico

Esta sección describe, para un desarrollador que se incorpore al proyecto, la responsabilidad de cada carpeta del código fuente.

Carpeta / archivo	Responsabilidad
src/config/database.js	Configuración y pool único de conexión a SQL Server (sql.ConnectionPool). Valida variables de entorno obligatorias, prohíbe la cuenta sa y expone getPool()/closePool().
src/controllers/	Un controlador por dominio (auth, venta, auditoria). Valida la forma de la solicitud HTTP, traduce errores de SQL a mensajes seguros para el usuario y arma la respuesta (render EJS o JSON), sin contener reglas de negocio.
src/middleware/authMiddleware.js	requireAuth (exige sesión activa) y guestOnly (impide reingresar al login ya autenticado).
src/routes/	Define los endpoints HTTP de cada dominio, aplica el middleware de sesión/permiso correspondiente y desactiva el caché de respuesta.
src/services/	Única capa que invoca al driver mssql. Construye parámetros tipados y el Table-Valued Parameter de ventas; no valida reglas de negocio, solo transporta datos hacia y desde los procedimientos almacenados.
src/views/	Plantillas EJS (login, dashboard, facturacion, auditoria) que renderizan HTML a partir de datos ya validados; usan escape automático de EJS para evitar HTML/XSS en campos como nombre de cliente o usuario.
src/public/	Activos estáticos: hoja de estilos corporativa (css/styles.css), script de facturación del navegador (js/facturacion.js) e imágenes.
database/	Scripts SQL numerados de instalación modular, el script consolidado SecureFinanceERP_Full.sql, y database/tests/ con las cuatro suites SQL más la prueba de integración de Node (TransactionIntegrationTests.js).
tests/	Pruebas de Node (node --test) con mocks del driver SQL, organizadas por dominio: auth, venta, facturacion (frontend) y audit.
server.js	Punto de entrada: crea la aplicación Express (createApp), configura sesión, monta las rutas de los tres dominios, define /health y el manejador de errores central, y arranca el servidor solo después de validar la conexión a SQL Server.

21. Limitaciones

Las siguientes limitaciones se identificaron por revisión directa del código y están documentadas también en el README del proyecto; se listan aquí sin minimizarlas ni exagerarlas.

- Recuperación y cambio de contraseña incompletos: existen el esquema Token_Recuperacion y el indicador DebeCambiarPassword, pero ninguna ruta ni vista de la aplicación web implementa el flujo de solicitud, verificación de token o cambio efectivo de contraseña.
- Sesiones en memoria: express-session usa MemoryStore, documentado como exclusivo para desarrollo académico; las sesiones se pierden al reiniciar el proceso y no escalan a múltiples instancias. Un despliegue real requiere un almacén de sesiones persistente.
- HTTPS no configurado: la aplicación escucha en HTTP sobre 127.0.0.1:3000; no incluye certificado ni configuración de proxy de confianza para un despliegue en producción.
- Administración de usuarios/roles solo por SQL: el permiso USUARIOS_ADMINISTRAR existe en el catálogo, pero no hay controlador ni vista web que lo consuma; la gestión de usuarios, roles y permisos se realiza ejecutando procedimientos o scripts directamente en SQL Server.
- Usuario DEMO académico: las credenciales admin_demo / Demo_Academica_2026! son exclusivamente de demostración local y no deben usarse ni reproducirse fuera de un entorno académico.
- Ejecución local: el sistema está pensado para ejecutarse localmente (Node.js + SQL Server en la misma red de desarrollo), sin balanceo de carga, contenedores ni infraestructura de alta disponibilidad.

22. Conclusiones

- La seguridad del sistema no depende de la capa de presentación: contraseñas, sales, verificación de identidad y resolución de permisos ocurren íntegramente dentro de SQL Server, y Node.js nunca observa ni transporta un hash o una sal.
- La integridad del negocio está garantizada por restricciones declarativas (CHECK, UNIQUE, FOREIGN KEY) además de la lógica procedural, de modo que un error en un procedimiento almacenado no podría, por ejemplo, dejar un Total distinto de Subtotal + IVA.
- El modelo RBAC de tres niveles (Usuario → Rol → Permiso) permite añadir nuevos roles o permisos sin modificar código de la aplicación, y se aplica de forma consistente tanto en el procedimiento de autenticación como en cada ruta protegida.
- Las propiedades ACID se demuestran de forma concreta y verificable en sp_ProcesarVentaTransaccional: una venta multiproducto con una sola línea sin stock revierte íntegramente, sin dejar facturas, detalles ni movimientos de caja parciales.
- El control de concurrencia mediante UPDLOCK/HOLDLOCK y el recorrido en orden ascendente por ProductoId reducen de forma medible el riesgo de condiciones de carrera e interbloqueo entre ventas simultáneas sobre los mismos productos, sin necesidad de reintentos automáticos que pudieran duplicar una venta.
- La auditoría automática por triggers captura cambios de negocio (precio, stock, eliminación de productos, emisión de facturas) sin depender de que la aplicación recuerde registrar cada operación, lo que la hace resistente a errores u omisiones del lado de Node.js.
- La integración de cuatro módulos desarrollados por integrantes distintos —seguridad, negocio/transacciones, auditoría y documentación— sobre un modelo de datos común fue el principal reto de coordinación del proyecto, y el historial de ramas e integración por pull request documenta cómo se resolvió.

- El aprendizaje técnico central del curso —que la base de datos, no la aplicación, es la autoridad de integridad y seguridad de un sistema financiero— queda demostrado de forma práctica en cada uno de los módulos descritos en este documento.

23. Recomendaciones

- Completar el flujo de recuperación/cambio de contraseña usando el esquema Token_Recuperacion ya existente, antes de considerar el módulo de seguridad apto para un entorno distinto al académico.
- Sustituir MemoryStore por un almacén de sesiones persistente (por ejemplo, una tabla propia en SQL Server o un almacén dedicado) si el sistema fuera a ejecutarse con más de un proceso o a sobrevivir reinicios.
- Configurar HTTPS y la bandera trust proxy correspondiente antes de exponer el sistema fuera de una red local de desarrollo.
- Construir una interfaz web para el permiso USUARIOS_ADMINISTRAR, de modo que la administración de usuarios, roles y permisos no dependa de ejecutar scripts SQL manualmente.
- Ejecutar y documentar formalmente, con evidencia en video o captura, las cuatro suites SQL (database/tests/) y npm run check:db contra una instancia real de SQL Server antes de la defensa final del proyecto, dado que este documento se preparó sin acceso a una instancia SQL Server activa.
- Mantener la disciplina ya presente en el proyecto de nunca versionar el archivo .env ni credenciales reales, y de conservar 10_AppPermissions.sql como única fuente de verdad de los permisos de la cuenta de aplicación.

Documentación técnica y funcional · ampliación integral

Se desarrollan las funciones descritas en los documentos de la rama revisada y en los diagramas compartidos. La evidencia de laboratorio se distingue de la aceptación en producción.

1. Ampliación de alcance

El proyecto conecta catálogo/proveedores, recepción de compra, inventario/Kardex, facturación, caja, posventa, garantía, clientes y auditoría. Node/Express controla la interacción, mientras SQL Server valida autorizaciones, reglas e integridad.

2. Flujo integrado de la operación

Cadena	Documentos/entidades	Resultado/control
Producto–proveedor	ProductoProveedor	Varios proveedores; principal/activo; no altera stock.
Compra–Kardex	RecepcionCompra, DetalleRecepcionCompra, Kardex_Inventario.	Documento/actor/cantidad; costo opcional; saldo por movimiento confirmado.
Venta–caja	Factura, DetalleFactura, MovimientoCaja, Kardex_Inventario.	Transacción ACID; operación idempotente ante reintento idéntico.
Emisión–historial	SolicitudVentaERP, FacturaProductoEmision.	Clave/huella para no duplicar; snapshot SKU/descripción de factura nueva.
Posventa	DevolucionCliente, DetalleDevolucion, NotaCredito, AplicacionNotaCredito, DevolucionKardex.	Cantidad y saldo validados; reembolso externo; reingreso condicionado.
Garantía	CoberturaGarantiaVenta, Ticket_Garantia, Historial_Garantia.	Ticket conectado a línea vendida y seguimiento por actor/estado.
Reactivación	ReactivacionCliente.	Cliente inactivo, motivo, permiso, ROWVERSION e historial inmutable.

3. Migraciones de datos 33–40

33 incorpora varios proveedores por producto. 34 agrega recepciones inmutables que afectan inventario por Kardex. 35 registra aplicaciones de notas de crédito y saldos como libro inmutable. 36 reúne permisos de integración. 37 protege reintentos de venta con OperacionId y huella del contenido. 38 genera SKU con secuencia SQL en vez de MAX+1. 39 captura SKU/descripción al emitir y habilita historial de factura. 40 reactiva clientes bajo motivo, permiso, control de versión y auditoría.

Las migraciones recientes no deben confundirse con instalar el paquete Full sobre una base existente. La instalación Full es para base nueva; las actualizaciones requieren migraciones incrementales coordinadas y respaldo verificable.

4. Reglas y significado

Factura, líneas, caja y Kardex confirman juntos o se revierten. Kardex es libro central; recepción, aplicaciones de crédito, tickets, snapshots, solicitudes y reactivaciones conservan eventos históricos. Duplicados NIT/SKU se rechazan; ROWVERSION evita sobrescrituras concurrentes. Líneas discontinuas del ERD indican auditoría/eventos y no necesariamente FK.

5. Estado de evidencia

La matriz recibida registra 95 PASS de Kardex/venta en entorno aislado, 79 PASS en una corrida Node, y pruebas separadas de compras, SKU, idempotencia, facturas, devoluciones, créditos, garantías, clientes y permisos. No sumar esos conteos. Parte del navegador usa una sesión fixture y no prueba login real; el login manual queda pendiente. Los resultados corresponden a laboratorio y el corte de estabilización indica producción no migrada. M40 se debe volver a aceptar en el commit destino.

El sistema documenta la referencia de reembolso externo, no transferencia bancaria; ventas brutas no descuentan crédito/devolución; no se acredita integración SAT/FEL ni prueba general de carga.

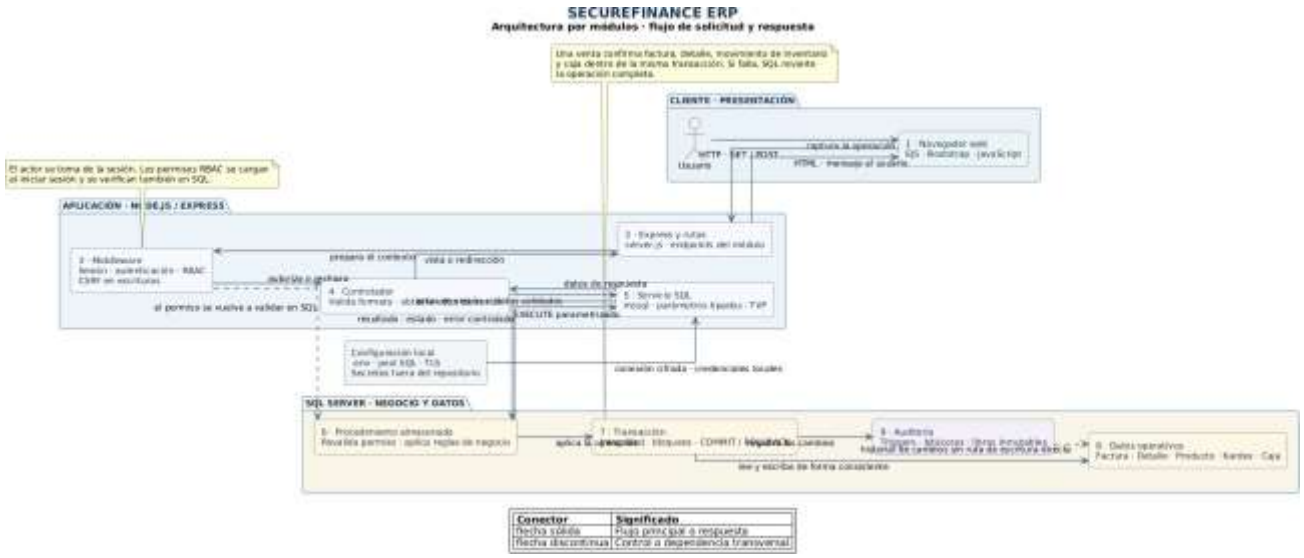


Figura G. Arquitectura modular actualizada.

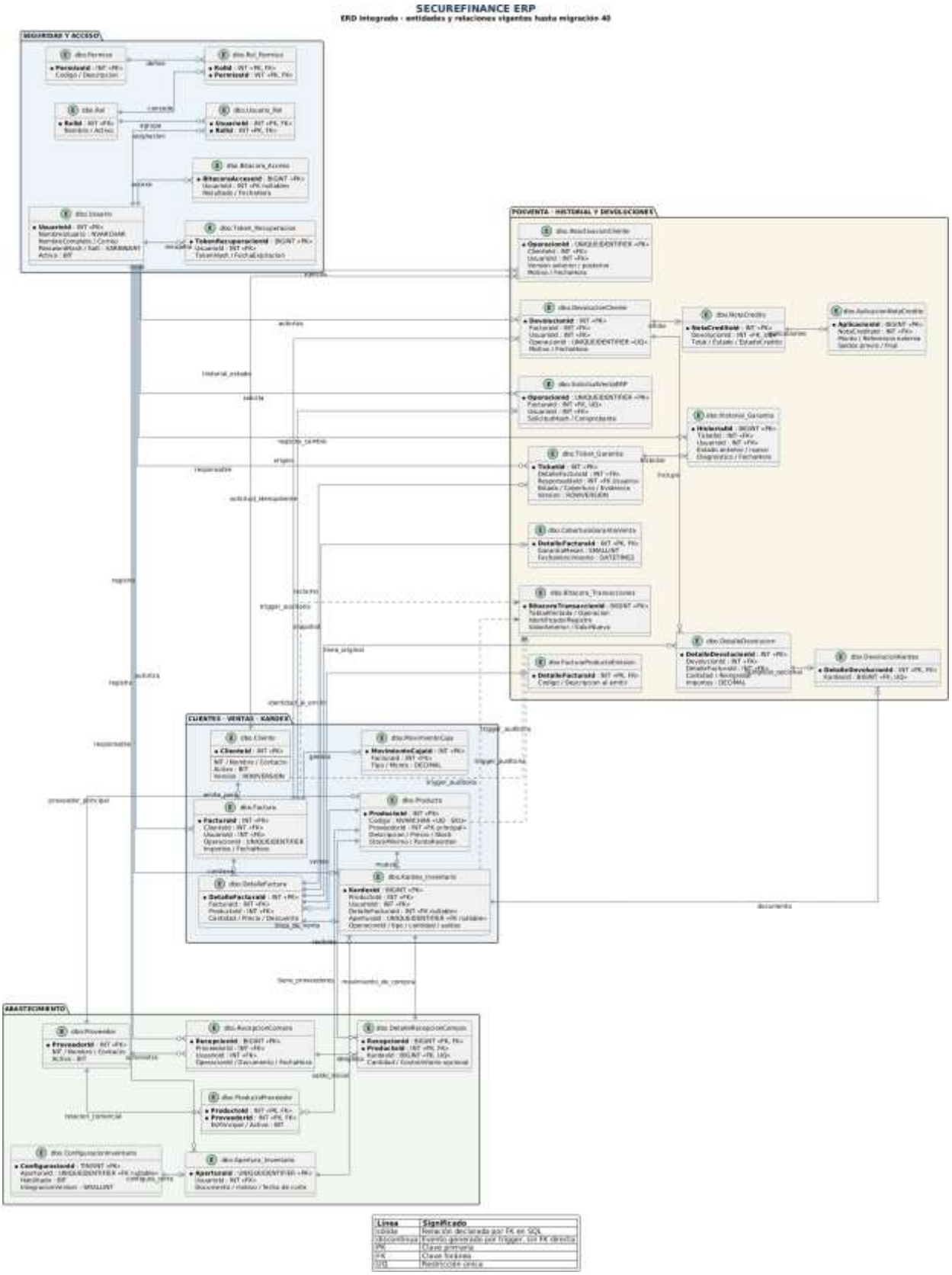


Figura H. ERD con módulos nuevos hasta migración 40.

Código PlantUML · Arquitectura_SecureFinance_ERP.puml

```
@startuml SecureFinance_ERP_Arquitectura
' Arquitectura lógica organizada por módulos y conectada por el flujo de una solicitud ' Basado en feature/estabilizacion-
funcional-erp · commit 224164a
left to right direction
skinparam backgroundColor #FFFFFF skinparam
shadowing false skinparam linetype ortho
skinparam roundcorner 8
skinparam defaultFontName Aptos skinparam
defaultFontSize 12
skinparam nodesep 36
skinparam ranksep 48 skinparam
ArrowColor #526B80 skinparam
ArrowThickness 1.4 skinparam rectangle
{
    BackgroundColor #F6F9FC
    BorderColor #31536F FontColor
    #17324D
}
skinparam package { BorderColor
    #8BA6BB FontColor #17324D
    FontStyle bold
}
title <size:20><color:#17324D>SECUREFINANCE ERP</color></size>\n<size:14>Arquitectura por módulos · flujo de solicitud y
respuesta</size>

package "CLIENTE · PRESENTACIÓN" #EAF2F8 {
    actor "Usuario" as Operador
    rectangle "1 · Navegador web\nEJS · Bootstrap · JavaScript" as Browser #EAF5F2
}

package "APLICACIÓN · NODE.JS / EXPRESS" #EDF5FA {
    rectangle "2 · Express y rutas\nserver.js · endpoints del módulo" as Express rectangle "3 · Middleware\nSesión ·
    autenticación · RBAC\nCSRF en escrituras" as
Middleware
    rectangle "4 · Controlador\nValida formato · obtiene actor de sesión" as Controller rectangle "5 · Servicio SQL\nmssql ·
    parámetros tipados · TVP" as Service
}

package "SQL SERVER · NEGOCIO Y DATOS" #F8F4E9 {
    rectangle "6 · Procedimiento almacenado\nRevalida permiso · aplica reglas de negocio" as Procedure #FFF6E5
    rectangle "7 · Transacción\nIntegridad · bloqueos · COMMIT / ROLLBACK" as Transaction #FFF6E5
    rectangle "8 · Datos operativos\nFactura · Detalle · Producto · Kardex · Caja" as Tables #FFF6E5
    rectangle "9 · Auditoría\nTriggers · bitácoras · libros inmutables" as Audit #F4EFF8
}
```

```
rectangle "Configuración local\n.env · pool SQL · TLS\nSecretos fuera del repositorio" as Config #F2F4F7

' Secuencia principal: solicitud desde la interfaz hasta la persistencia Operador --> Browser : captura la
operación
Browser --> Express : HTTP · GET / POST Express -->
Middleware : prepara el contexto Middleware --> Controller :
autoriza o rechaza
Controller --> Service : actor de sesión + datos validados Service --> Procedure :
EXECUTE parametrizado
Procedure --> Transaction : aplica la operación
Transaction --> Tables : lee y escribe de forma consistente Transaction --> Audit : registra
los cambios

' Respuesta del servidor hacia la interfaz
Procedure --> Service : resultado · estado · error controlado Service --> Controller : datos
de respuesta
Controller --> Express : vista o redirección Express --> Browser : HTML ·
mensaje al usuario

' Dependencias que cruzan módulos
Config --> Service : conexión cifrada · credenciales locales Middleware ..> Procedure : el permiso se
vuelve a validar en SQL Audit ..> Tables : historial de cambios sin ruta de escritura directa

note bottom of Transaction
    Una venta confirma factura, detalle, movimiento de inventario y caja dentro de la misma
    transacción. Si falla, SQL revierte la operación completa.
end note

note bottom of Middleware
    El actor se toma de la sesión. Los permisos RBAC se cargan al iniciar sesión y se
    verifican también en SQL.
end note

legend bottom
    |= Conector |= Significado |
    | flecha sólida | Flujo principal o respuesta |
    | flecha discontinua | Control o dependencia transversal | endlegend
@enduml
```

Código PlantUML · ERD_SecureFinance_ERP.puml

```
@startuml SecureFinance_ERP_ERD
' ERD de SecureFinance ERP basado en las claves del DDL de la rama ' feature/estabilizacion-
funcional-erp · commit 224164a
left to right direction
skinparam backgroundColor #FFFFFF skinparam
shadowing false skinparam linetype ortho
skinparam roundcorner 8
skinparam defaultFontName Aptos skinparam
defaultFontSize 12
skinparam nodesep 36
skinparam ranksep 48 skinparam
ArrowColor #526B80 skinparam
ArrowThickness 1.25 skinparam entity {
    BackgroundColor #F6F9FC
    BorderColor #31536F FontColor
    #17324D
    HeaderBackgroundColor #17324D
    HeaderFontColor #FFFFFF
    AttributeFontColor #263746
    AttributeFontSize 10
}
title <size:20><color:#17324D>SECUREFINANCE ERP</color></size>\n<size:14>ERD integrado · entidades y relaciones vigentes hasta
migración 40</size>

package "CLIENTES · VENTAS · KARDEX" #EAF2F8 {
' Flujo central del modelo: cliente □ factura □ detalle □ producto □ inventario entity "dbo.Cliente" as Cliente {
    * **ClienteId** : INT <<PK>>
    --
    NIT / Nombre / Contacto Activo : BIT
    Version : ROWVERSION
}
entity "dbo.Factura" as Factura {
    * **FacturaId** : INT <<PK>> ClienteId :
    INT <<FK>> UsuarioId : INT <<FK>>
    OperacionId : UNIQUEIDENTIFIER
    Importes / FechaHora
}
entity "dbo.DetalleFactura" as DetalleFactura {
    * **DetalleFacturaId** : INT <<PK>> FacturaId :
    INT <<FK>>
    ProductoId : INT <<FK>>
```

```
    Cantidad / Precio / Descuento
}
entity "dbo.Producto" as Producto {
    * **ProductoId** : INT <<PK>> Codigo :
    NVARCHAR <<UQ · SKU>>
    ProveedorId : INT <<FK principal>> Descripcion /
    Precio / Stock StockMinimo / PuntoReorden
}
entity "dbo.Kardex_Inventario" as Kardex {
    * **KardexId** : BIGINT <<PK>>
    ProductoId : INT <<FK>> UsuarioId : INT
    <<FK>>
    DetalleFacturaId : INT <<FK nullable>> AperturaId :
    UNIQUEIDENTIFIER <<FK nullable>> OperacionId / tipo / cantidad /
    saldos
}
entity "dbo.MovimientoCaja" as MovimientoCaja {
    * **MovimientoCajaId** : INT <<PK>> FacturaId
    : INT <<FK>>
    Tipo / Monto : DECIMAL
}

}

package "SEGURIDAD Y ACCESO" #EDF5FA {
' Las tablas puente forman una sola cadena RBAC entity "dbo.Usuario" as
Usuario {
    * **UsuarioId** : INT <<PK>>
    NombreUsuario : NVARCHAR
    NombreCompleto / Correo PasswordHash /
    Salt : VARBINARY Activo : BIT
}
entity "dbo.Usuario_Rol" as UsuarioRol {
    * **UsuarioId** : INT <<PK, FK>>
    * **RolId** : INT <<PK, FK>>
}
entity "dbo.Rol" as Rol {
    * **RolId** : INT <<PK>>
    Nombre / Activo
}
entity "dbo.Rol_Permiso" as RolPermiso {
    * **RolId** : INT <<PK, FK>>
    * **PermisoId** : INT <<PK, FK>>
```

```
}
entity "dbo.Permiso" as Permiso {
    * **PermisoId** : INT <<PK>> Codigo
    / Descripcion
}
entity "dbo.Bitacora_Acceso" as BitacoraAcceso {
    * **BitacoraAccesoId** : BIGINT <<PK>> UsuarioId
    : INT <<FK nullable>> Resultado / FechaHora
}
entity "dbo.Token_Recuperacion" as TokenRecuperacion {
    * **TokenRecuperacionId** : BIGINT <<PK>> UsuarioId
    : INT <<FK>>
    TokenHash / FechaExpiracion
}

}

package "ABASTECIMIENTO" #F2F7F1 {
' El proveedor entrega un producto y su recepción crea Kardex entity "dbo.Proveedor" as
Proveedor {
    * **ProveedorId** : INT <<PK>> NIT /
    Nombre / Contacto
    Activo : BIT
}
entity "dbo.ProductoProveedor" as ProductoProveedor {
    * **ProductoId** : INT <<PK, FK>>
    * **ProveedorId** : INT <<PK, FK>>
    EsPrincipal / Activo : BIT
}
entity "dbo.RecepcionCompra" as RecepcionCompra {
    * **RecepcionId** : BIGINT <<PK>>
    ProveedorId : INT <<FK>> UsuarioId : INT
    <<FK>>
    OperacionId / Documento / FechaHora
}
entity "dbo.DetalleRecepcionCompra" as DetalleRecepcion {
    * **RecepcionId** : BIGINT <<PK, FK>>
    * **ProductoId** : INT <<PK, FK>> KardexId
    : BIGINT <<FK, UQ>>
    Cantidad / CostoUnitario opcional
}
entity "dbo.Apertura_Inventario" as AperturaInventario {
    * **AperturaId** : UNIQUEIDENTIFIER <<PK>>
    UsuarioId : INT <<FK>>
```

```
    Documento / motivo / fecha de corte
}
entity "dbo.ConfiguracionInventario" as ConfiguracionInventario {
    * **ConfiguracionId** : TINYINT <<PK>> AperturaId :
    UNIQUEIDENTIFIER <<FK nullable>> Habilitado : BIT
    IntegracionVersion : SMALLINT
}

}

package "POSVENTA · HISTORIAL Y DEVOLUCIONES" #F8F4E9 {
entity "dbo.SolicitudVentaERP" as SolicitudVenta {
    * **OperacionId** : UNIQUEIDENTIFIER <<PK>>
    FacturaId : INT <<FK, UQ>>
    UsuarioId : INT <<FK>>
    SolicitudHash / Comprobante
}
entity "dbo.FacturaProductoEmision" as ProductoEmision {
    * **DetalleFacturaId** : INT <<PK, FK>> Codigo /
    Descripcion al emitir
}


entity "dbo.DevolucionCliente" as Devolucion {
    * **DevolucionId** : INT <<PK>>
    FacturaId : INT <<FK>> UsuarioId : INT
    <<FK>>
    OperacionId : UNIQUEIDENTIFIER <<UQ>>
    Motivo / FechaHora
}
entity "dbo.DetalleDevolucion" as DetalleDevolucion {
    * **DetalleDevolucionId** : INT <<PK>>
    DevolucionId : INT <<FK>> DetalleFacturaId : INT
    <<FK>>
    Cantidad / Reingresar Importes :
    DECIMAL
}
entity "dbo.NotaCredito" as NotaCredito {
    * **NotaCreditoId** : INT <<PK>>
    DevolucionId : INT <<FK, UQ>> Total /
    Estado / EstadoCredito
}
entity "dbo.AplicacionNotaCredito" as AplicacionNotaCredito {
    * **AplicacionId** : BIGINT <<PK>>
```

```
    NotaCreditoId : INT <<FK>> Monto /
    Referencia externa Saldos previo /
    final
}
entity "dbo.DevolucionKardex" as DevolucionKardex {
    * **DetalleDevolucionId** : INT <<PK, FK>> KardexId :
    BIGINT <<FK, UQ>>
}


entity "dbo.CoberturaGarantiaVenta" as CoberturaGarantia {
    * **DetalleFacturaId** : INT <<PK, FK>>
    GarantiaMeses : SMALLINT FechaVencimiento :
    DATETIME2
}
entity "dbo.Ticket_Garantia" as TicketGarantia {
    * **TicketId** : INT <<PK>> DetalleFacturaId
    : INT <<FK>> ResponsableId : INT <<FK
    Usuario>> Estado / Cobertura / Evidencia
    Version : ROWVERSION
}
entity "dbo.Historial_Garantia" as HistorialGarantia {
    * **HistorialId** : BIGINT <<PK>> TicketId :
    INT <<FK>>
    UsuarioId : INT <<FK>> Estado
    anterior / nuevo Diagnostico /
    FechaHora
}
entity "dbo.ReactivacionCliente" as ReactivacionCliente {
    * **OperacionId** : UNIQUEIDENTIFIER <<PK>>
    ClienteId : INT <<FK>>
    UsuarioId : INT <<FK>> Version anterior /
    posterior Motivo / FechaHora
}


entity "dbo.Bitacora_Transacciones" as BitacoraTransacciones {
    * **BitacoraTransaccionId** : BIGINT <<PK>>
    TablaAfectada / Operacion IdentificadorRegistro
    ValorAnterior / ValorNuevo
}
```

```
}

' Relaciones físicas FK: los conectores atraviesan módulos y conservan la secuencia operativa
Cliente ||--o{ Factura : emite_para Usuario ||--o{ Factura :
registra Factura ||--o{ DetalleFactura : contiene
DetalleFactura }o--|| Producto : vende Factura ||--o{
MovimientoCaja : genera Producto ||--o{ Kardex : mueve
Usuario ||--o{ Kardex : autoriza DetalleFactura ||--o| Kardex :
linea_de_venta

' Seguridad conectada al actor del proceso Usuario ||--o{ UsuarioRol :
asignacion
Rol ||--o{ UsuarioRol : agrupa Rol ||--o{
RolPermiso : concede
Permiso ||--o{ RolPermiso : define Usuario ||--o{
BitacoraAcceso : acceso
Usuario ||--o{ TokenRecuperacion : recupera Usuario ||--o{
ReactivacionCliente : ejecuta
Cliente ||--o{ ReactivacionCliente : historial_estado

' Abastecimiento conectado con los mismos productos y el Kardex central Proveedor ||--o{
ProductoProveedor : relacion_comercial
Producto ||--o{ ProductoProveedor : tiene_proveedores Proveedor ||--o{ Producto :
proveedor_principal Proveedor ||--o{ RecepcionCompra : suministra
Usuario ||--o{ RecepcionCompra : registra RecepcionCompra ||--o{
DetalleRecepcion : desglosa Producto ||--o{ DetalleRecepcion : recibido
DetalleRecepcion ||--|| Kardex : movimiento_de_compra AperturaInventario ||--o{
Kardex : saldo_inicial
ConfiguracionInventario ||--o| AperturaInventario : configura_corte Usuario ||--o{ AperturaInventario :
responsable

' Consulta histórica e idempotencia alrededor de la factura persistida Factura ||--o| SolicitudVenta :
solicitud_idempotente
Usuario ||--o{ SolicitudVenta : solicita
DetalleFactura ||--o| ProductoEmision : identidad_al_emitir

' Posventa enlazada a factura, línea original y movimientos de inventario Factura ||--o{ Devolucion : origen
Usuario ||--o{ Devolucion : autoriza Devolucion ||--o{
DetalleDevolucion : incluye
```

```
DetalleFactura ||--o{ DetalleDevolucion : linea_original Devolucion ||--o|
NotaCredito : emite
NotaCredito ||--o{ AplicacionNotaCredito : aplicaciones DetalleDevolucion ||--o|
DevolucionKardex : reingreso_opcional Kardex ||--o| DevolucionKardex : documenta

' Garantía sigue la línea original, no el catálogo actual DetalleFactura ||--o|
CoberturaGarantia : snapshot DetalleFactura ||--o{ TicketGarantia : reclamo
Usuario ||--o{ TicketGarantia : responsable TicketGarantia ||--o{ HistorialGarantia :
historial Usuario ||--o{ HistorialGarantia : registra_cambio

' Eventos de auditoría son líneas discontinuas porque no representan FK Cliente ..> BitacoraTransacciones :
trigger_auditoria
Producto ..> BitacoraTransacciones : trigger_auditoria Factura ..>
BitacoraTransacciones : trigger_auditoria Kardex ..> BitacoraTransacciones :
trigger_auditoria

' Anclas invisibles: conservan el orden operativo de las entidades centrales Cliente -[hidden]right- Factura
Factura -[hidden]right- DetalleFactura DetalleFactura -[hidden]right-
Producto Producto -[hidden]right- Kardex

legend bottom
    |= Línea |= Significado |
    | sólida | Relación declarada por FK en SQL |
    | discontinua | Evento generado por trigger, sin FK directa |
    | PK | Clave primaria |
    | FK | Clave foránea |
    | UQ | Restricción única | endlegend
@enduml
```